

SSR 2

- [Проблемы SSR](#)
 - [Render Performance](#)
 - [Nodejs](#)
 - [Очередь запросов](#)
 - [Как тогда масштабироваться?](#)
 - [Скейлинг \(Горизонтальное масштабирование\)](#)
 - [Kubernetes + Load balancer.](#)
 - [Стоимость железа](#)
 - [Инфраструктура](#)
 - [Деплой](#)
 - [Сложность как итог](#)
- [Оптимизация](#)
 - [Оптимизация SSR](#)
 - [Хак для SEO](#)
 - [Альтернативные режимы рендера](#)
 - [SSG \(Static Site Generation\)](#)
 - [Основные минусы SSG](#)
 - [Когда SSG — не лучший выбор?](#)
 - [Edge Rendering](#)
 - [Static проблемы](#)
 - [SSG Fraemworks](#)
 - [Partial SSG \(частичный SSG\)](#)
 - [CSR + SSG + SSR](#)
 - [App Shell](#)
 - [ISR \(Incremental Static Regeneration\)](#)
 - [SSI \(Server Side Include\)](#)
- [Оптимизация процесса SSR](#)
 - [Из чего складывается время рендера страницы](#)
 - [Кэширование](#)
 - [Кэширование запросов](#)
 - [Кэширование компонентов](#)
 - [Partial prerendering](#)
 - [Разделяемый Кэш](#)

- [Кэширование уровнем выше](#)
- [Потоковый рендер](#)
- [Предзагрузка/инлайн ассетов](#)
- [Оптимизация гидрации](#)
 - [Partial Hydration](#)
 - [Lazy Hydration](#)
 - [Progressive Hydration](#)
 - [Early Hints](#)
- [Альтернативные подходы SSR](#)
 - [HTMX](#)
 - [HotWite](#)
 - [Qwik](#)
 - [Astro](#)
 - [Marko](#)

Проблемы SSR

Render Performance

Это самая важная проблема на сегодняшний день у SSR.

Основная мера величины перформанса на бэкенде - это RPS

RPS - Request Per Second - Сколько запросов в секунду может обработать сервер.

Может обработать - это без жесткой деградации обработка.

Важно, что все это учитывает на одно ядро (так как это полезно при горизонтальном масштабировании). Также важная величина - частота ядра, особенно важно для Nodejs (т.к. он однопоточный), сейчас в дата-центрах многоядерные процессоры, с не очень высокой частотой.

- По факту цифры такие: 3-20 RPS при честном SSR. Full SSR + 95% in 1s .
- В Nextjs используется всякие хитрости, в том числе нечестный SSR, там больше + от Vercel есть хитрости.

Nodejs

Упираемся именно в Nodejs и в целом JavaScript. Изначально они оба используют движок V8. Она **однопоточная асинхронная**. Она работает в одном ядре и есть **EventLoop**.

В SSR сталкиваемся с проблемой, что данная очередь переполняется. Она очень-очень длинная. Так еще Nodejs может ходить в другие API, отсюда еще и обработка запросов, удержание конекшена и так далее.

Очередь запросов

- Есть Render, по сути вызов `renderToString`, занимает 50ms.
- Однако если у нас есть 2 RPS, тогда должны первый запрос обработать это 50ms , затем второй запрос, и вот второй запрос будет ждать первый и он уже будет 100ms и так далее.



Допустим, Event Loop сильно загружен: один только `render` занимает 70 мс, плюс добавляются сетевые запросы. В результате начинает расти так называемый **Event Loop Lag** — то есть задержка в обработке задач Event Loop'ом.

Из-за этого может возникнуть ситуация: в очереди уже накопилось много задач на рендеринг, и есть компонент, которому для рендера нужны данные. Но он «зависает» — просто не может получить эти данные, потому что Event Loop настолько занят, что не успевает обработать промис, который должен вернуть нужную информацию.

И SSR как раз живет вместе с микротасками очень тесно. Выход в том, чтобы менять архитектуру.

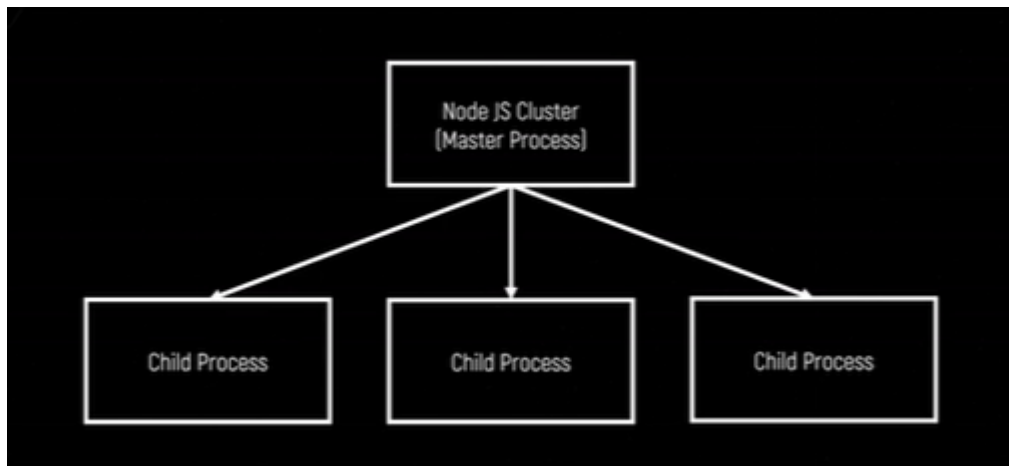
Как тогда масштабироваться?

Есть запросы, синхронный рендер, одно ядро может держать 20 RPS.

Скейлинг (Горизонтальное масштабирование)

Распараллеливаем процесс рендера.

- Самый простой вариант, взять модуль `NodeJS Cluserter` (Master Process) и запустить несколько инстансов NodeJS. Количество инстансов зависит от того, сколько ядер выделяют на виртуалку:



Проблема масштабирования:

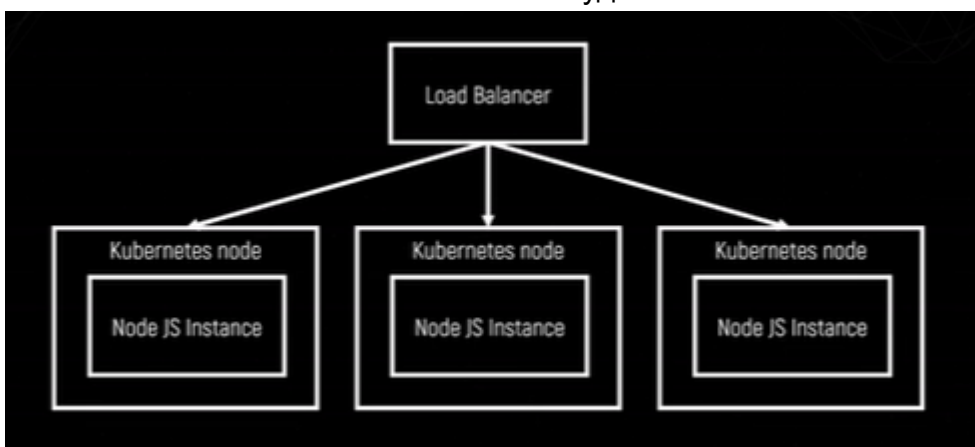
Когда мы начинаем масштабировать приложение, сильно увеличиваются накладные расходы. В случае с Node.js, если использовать кластеризацию (Cluster), то на одной виртуальной машине запускаются несколько процессов Node.js, а основной (master) процесс сам распределяет входящие запросы между ними.

Но у этой схемы есть недостаток: встроенный балансировщик нагрузки в Node.js работает неэффективно. При высокой нагрузке сам **master-процесс** начинает «тормозить» и не справляется с распределением задач. Поэтому Node.js Cluster подходит только для **простых и небольших** случаев — на серьёзных нагрузках он быстро становится узким местом.

Kubernetes + Load balancer.

Правильнее использовать Kubernetes, у которого сразу же есть балансировщик из коробки и балансировку выполняет сам кубер.

То есть "Load Balancer" - это какой-нибудь внешний механизм:



И уже эта схема добавляет много плюшек (доступен ли инстанс, авто-балансировка, выкидываем зависшего процесса и так далее). Но это стоит приличных денег и усложнения инфраструктуры.

Стоимость железа

Стоимость железа

10 RPS per core

~100 RPS

= 12 cores

6 VPS * 2 cores

Reverse proxy (nginx) + 1 core

Docker registry

S3 space + CDN

2 512,30 ₽ в месяц

[Тарифы и цены](#)

Intel Ice Lake. 100% vCPU

1 632,96 ₽

Публичный IP-адрес

186,62 ₽

Intel Ice Lake. RAM

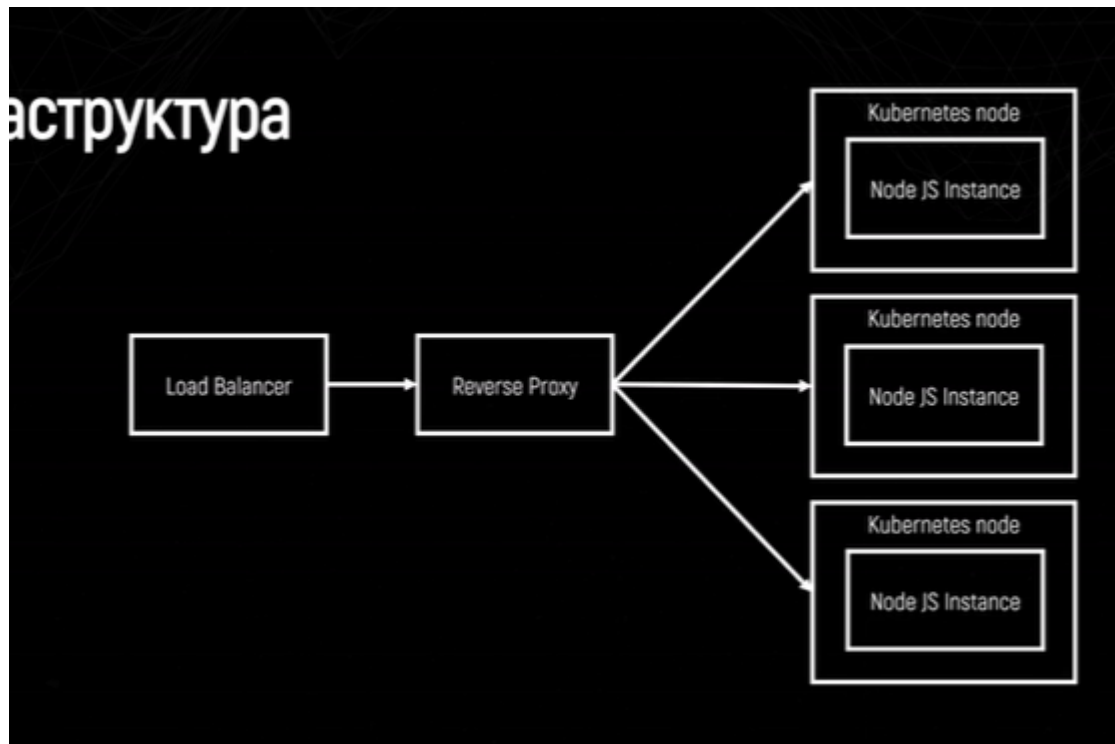
435,46 ₽

Быстрое сетевое хранилище (SSD)

257,26 ₽

Не считая Master Process Kubernetes

Инфраструктура



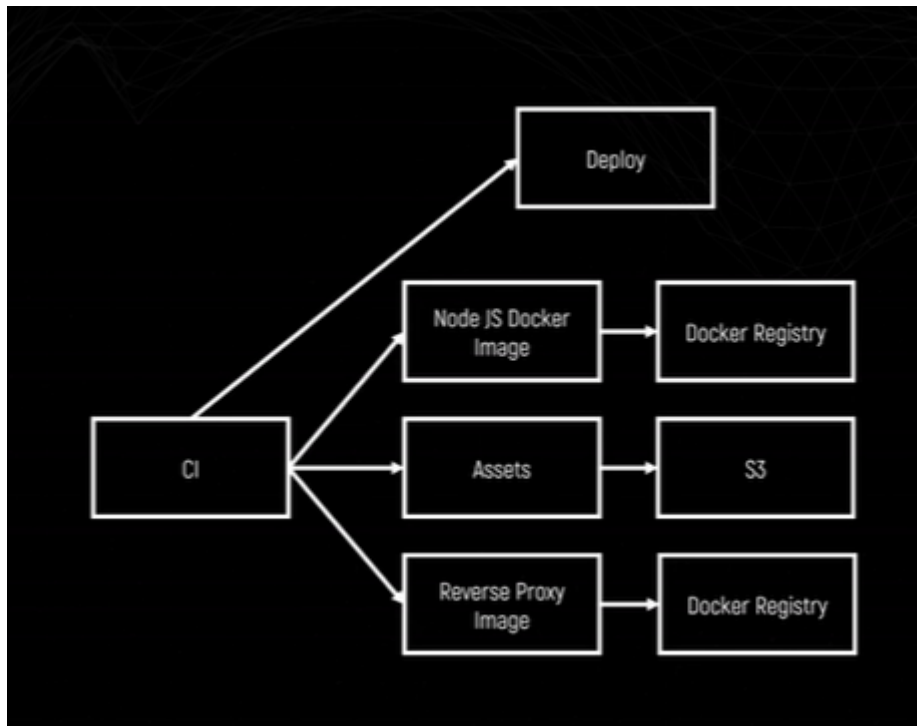
Примерно: есть

load balancer, который балансирует нагрузку, при чем он может быть встроен внутри в reverse proxy, может быть такое что reverse proxy одновременно является балансером, либо он отдельная сущность. (но здесь нужно помнить про отказоустойчивость).

Деплой

Есть NodeJS с серверным рендерингом + есть клиентский рендер, гидрация + клиентские ассеты + docker-образ ноды + docker-образ балансировщика, либо Reverse Proxy (т.к. на reverse proxy все равно навешивает часть задач по типу работа с https, обработка роутов типа robots.txt).

- По классике есть CI.
 - Запускаем сборку, на выходе получаем Assets (обычно клиентские ассеты).
 - Мы их деплоим на S3.
 - Помимо этого из CI должны собирать образ докера с NodeJS и с серверными ассетами, чтобы запускать это в рантайме.
 - И нужен Docker Registry куда мы это все деплоим.
 - Плюс есть Reverse Proxy тоже образ и его тоже деплоим.
- Хороший VPS - Timeweb cloud.



Сложность как итог

- Множество механизмов для реализации.
- Роутинг.
- Запросы / стейт (сериализация).
- SEO
- i18n
- Рендер времени пользователя (на сервере на первый рендер точно нельзя узнать какой часовой пояс у пользователя).
- critical CSS
- managing assets
- developer experience

Оптимизация

- Клиентский код все равно придется оптимизировать. Во-первых SSR не делает так, чтобы кода грузилось меньше. Там нужно доп оптимизации для этого делать. И после SSR это уже обычный клиентский рендер.
- Серверный код также необходимо оптимизировать: потребление ресурсов, организация и поддержка инфраструктуры, утечка памяти
- Поддержка нескольких режимов работы (сервер + useEffect).
- Тестирование в двух экземпляров (сервер + клиент).

Оптимизация SSR

- Идеально, чтобы Нода занималась рендером серверным, и на нее больше ничего не надо складывать. Не надо делать из нее еще API, SEO отдавала. Задача Ноду максимально разгрузить, чтобы она отвечала только за SSR. Выносить в отдельный сервис. Хотя Nextjs так делает, но если SSR самостоятельный, нужно все разделять.
- Отвечать на вопрос: действительно ли SSR сейчас нужен прямо сейчас?

Хак для SEO

Если нам нужен SSR чисто для SEO, а первая загрузка у нас хорошая. То можно сделать простую вещь: отдавать SSR только роботам, это возможно. Делается это по user-agent, когда приходят поисковые роботы индексировать сайт, они приходят со специальными user-agent'ом (Даже есть отдельные библиотеки, которые понимают когда пришел робот, а когда нет). И можно отдавать только SSR контент, а пользователям CSR.

В Яндексе даже есть отдельная инфраструктура: была Нода, которая с энвой только клиентский-рендер отдавала и был Кубер, который отправлял всех пользователей в эту Ноду, а роботов в SSR-ноду.

То есть это не экономия на разработке, не экономия на обучении людей, не экономия на ci/cd и кубера. Это именно экономия на железе.

- **Риск "Клоакинга"** (Cloaking) – если контент для роботов и пользователей сильно отличается, поисковики могут наказать.

Альтернативные режимы рендера

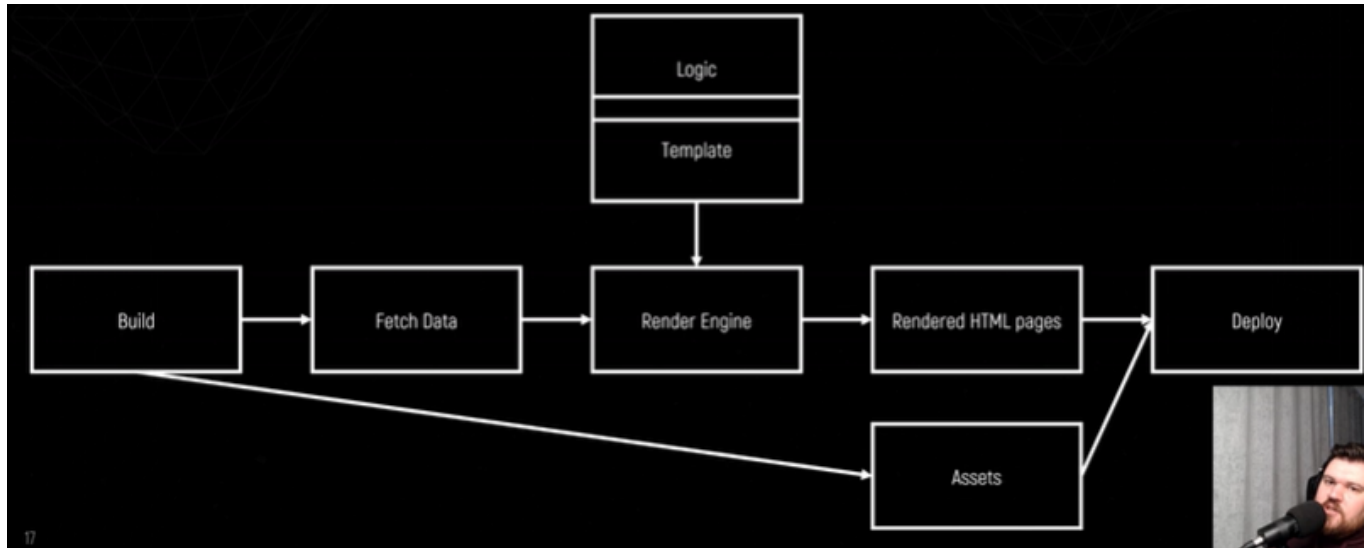
Чтобы понять, когда нужен альтернативный режим, нужно понять когда нужен SSR:

- SEO
- Performance (очень долгая первая отрисовка, куча js)
Чтобы понять, суть дальнейших оптимизаций и где можно отказаться от серверного рендеринга, это ответить на простой вопрос: **"В какой момент данные доступны и нужны?"**.

SSG (Static Site Generation)

Это метод сборки сайта, при котором страницы генерируются на этапе **билда** (во время сборки проекта), а не в рантайме (при запросе пользователя).

В результате получаются статические HTML-файлы, которые можно быстро отдавать с сервера или CDN



- Берем сборочку, она по дефолту билдит ассеты и заливает на S3 (деплой).
- **Параллельно!** Прямо во время сборки (или после сборки, это отдельный пайплайн). Мы запрашиваем данные.
- Отдаем их движку рендера вместе с логикой и шаблонами.
- На выходе получаем отрендеренный HTML-странички.
- И их тоже деплоем.

Все что можно показать заранее, мы уже зарендерили заранее.

У **SSG (Static Site Generation)** есть несколько минусов, которые стоит учитывать при выборе этого подхода:

Основные минусы SSG

1. Нет динамического контента "из коробки"

- Если данные часто меняются (например, актуальные цены, новости, личный кабинет), SSG требует пересборки всего сайта или части страниц.
- Решения: Incremental Static Regeneration (ISR в Next.js), клиентский fetch после загрузки страницы.

2. Долгая сборка на больших сайтах

- Если тысячи страниц (например, интернет-магазин или блог с огромной базой), билд может занимать **минуты или даже часы**.
- Решения: ISR, ленивая генерация, деление на подпроекты.

3. Нет доступа к runtime-данным при сборке

- Невозможно использовать данные, которые зависят от запроса пользователя (куки, геолокация, заголовки).
- Решения: комбинировать с CSR (Client-Side Rendering) или SSR для отдельных страниц.

4. Ограниченная интерактивность

- Если нужен сложный динамический интерфейс (например, чат, формы с валидацией), приходится добавлять JavaScript, что может снизить преимущества статики.

Когда SSG — не лучший выбор?

- ✗ Онлайн-магазин с живыми ценами и наличием товаров
- ✗ Приложение с персонализированным контентом (лента соцсети)
- ✗ Сайты, где контент обновляется каждые несколько минут

Edge Rendering

Edge Rendering — это способ обработки и доставки веб-контента, при котором рендеринг (создание HTML-страницы) происходит **не на сервере** (в одном центральном месте), а **на edge-серверах**, то есть на серверах, расположенных **ближе к пользователю географически**.

Пример технологии: Next.js Edge Functions, Cloudflare Workers

Недостатки:

1. непонятный статус, сейчас все хотят отказываться от него
2. невозможно разрабатывать локально, по сути это честное тестирование на продакшене (Edge-функции **запускаются в окружении, которое не имитируется локально на 100%**, Например, Cloudflare Workers использует V8 isolates с ограничениями, Vercel Edge Functions работают в Web Runtime (на базе V8), **не в полноценном Node.js**)
3. уже не наблюдается прирост перфомансе с развитием других гибридных рендерингов
Это решение должно решать только проблему перфоманса.

Static проблемы

- Весь контент должен быть статическим. Заранее на этапе билда необходимо получить все данные, вставить и вернуть готовый HTML. Зачастую полностью статических страниц ну наверно 2 (это о компании и контакты).
- Контент должен не зависеть от данных, который предоставляет входящий запрос. То есть не зависеть от контекста пользователя.(то есть делать страничку карточки товара

плохо делать)!!!

- Какие странички хорошо становятся статикой: документация, блог/статья, страница без динамики.

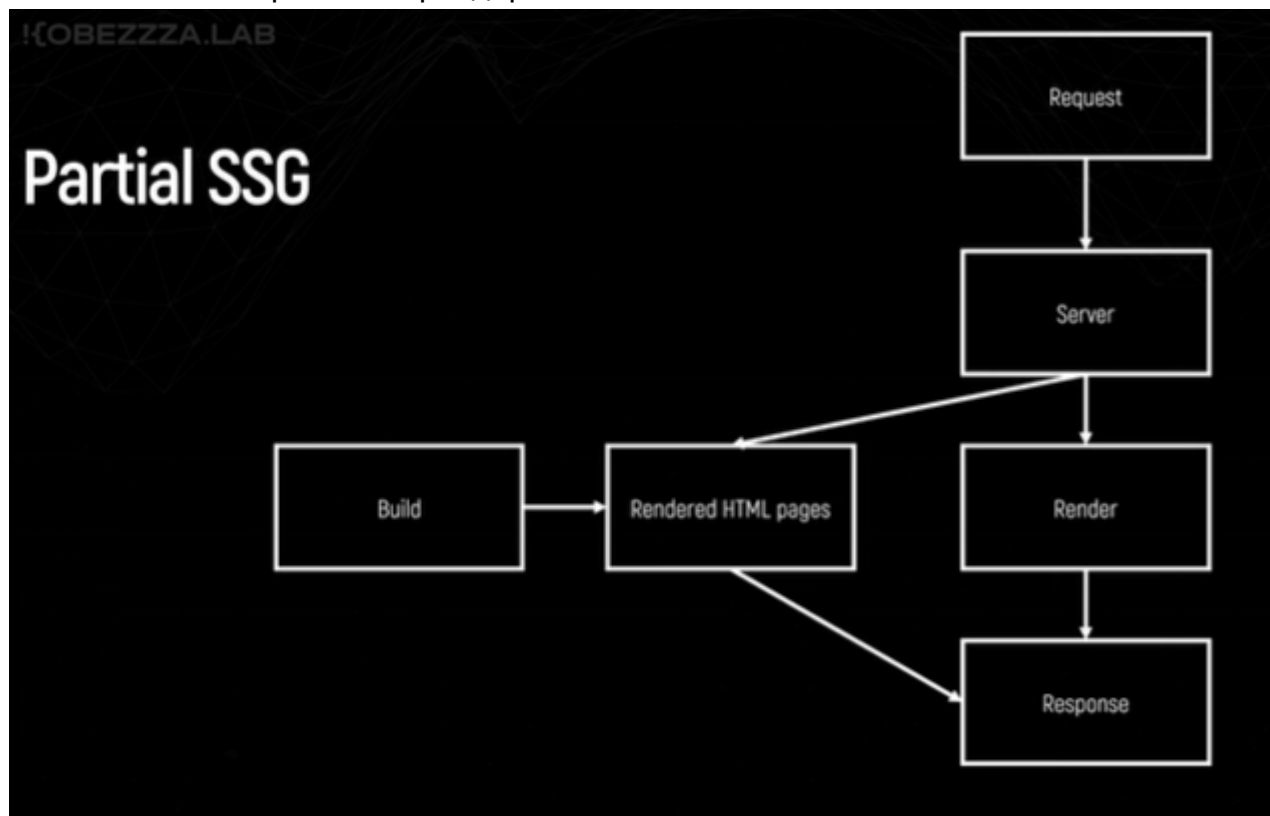
SSG Fraemworks

- Gatsby (старый, ушел в сторону netlify).
- Docusaurus (для генерации документации)
- VitePress
- Next.js
- Full List <https://jamstack.org/generators/>

Partial SSG (частичный SSG)

Никто не запрещает частично рендерить статику, миксовать это.

- Допустим на этапе билда точно также пререндерим и складываем как HTML.
- Когда приходит request на наш сервер, мы уже имеем возможность выбрать:
- Если есть заранее зарендеренная страничка - отдаем из кэша.
- Если нет такой странички - рендерим



CSR + SSG + SSR

- Забилдили - получили пререндеры HTML-странички

- Приходит также request, обрабатываем запрос на сервере
 - Теперь у нас три выбора:
1. Сначала смотрим в кэш на пререндер странички
 2. Если нету, значит смотрим в SSR ее отрендерить очень дорого и медленно, не получится.
 3. Значит отдаем Emtry CSR
 4. В итоге все равно отдаем ответ, контент.

На самом деле перформанс в вебе (говорим именно про рендера, то насколько пользователю приятно пользоваться страничкой) - это на самом деле очень **субъективная**. Да есть объективные метрики скорости. Но есть еще психологическое восприятие.

То есть если при SSR пользователь видит белый экран 3 секунды, то это плохо.

А вот если при клиентском рендере подождать с лоадерами 3 секунды - это вообще не проблема. То есть уже хоть какой то контент есть.

Идеальный пример - github, сам html он отдает очень долго, больше секунды, но при этом он очень быстро успевает отрендерить что-то вокруг: иконки, хедер, футер и так далее.

Поэтому ощущается и смотрится это быстро, хотя по факту по метрикам это очень медленно.

App Shell

Следующая важная концепция для оптимизации рендера - App Shell.

App Shell - это паттерн, который позволяет часть страницы отрендерить во время сборки или где-то заранее, , особенно в **SPA** и **PWA**.

Она помогает улучшить **время первого отображения (First Contentful Paint)**.

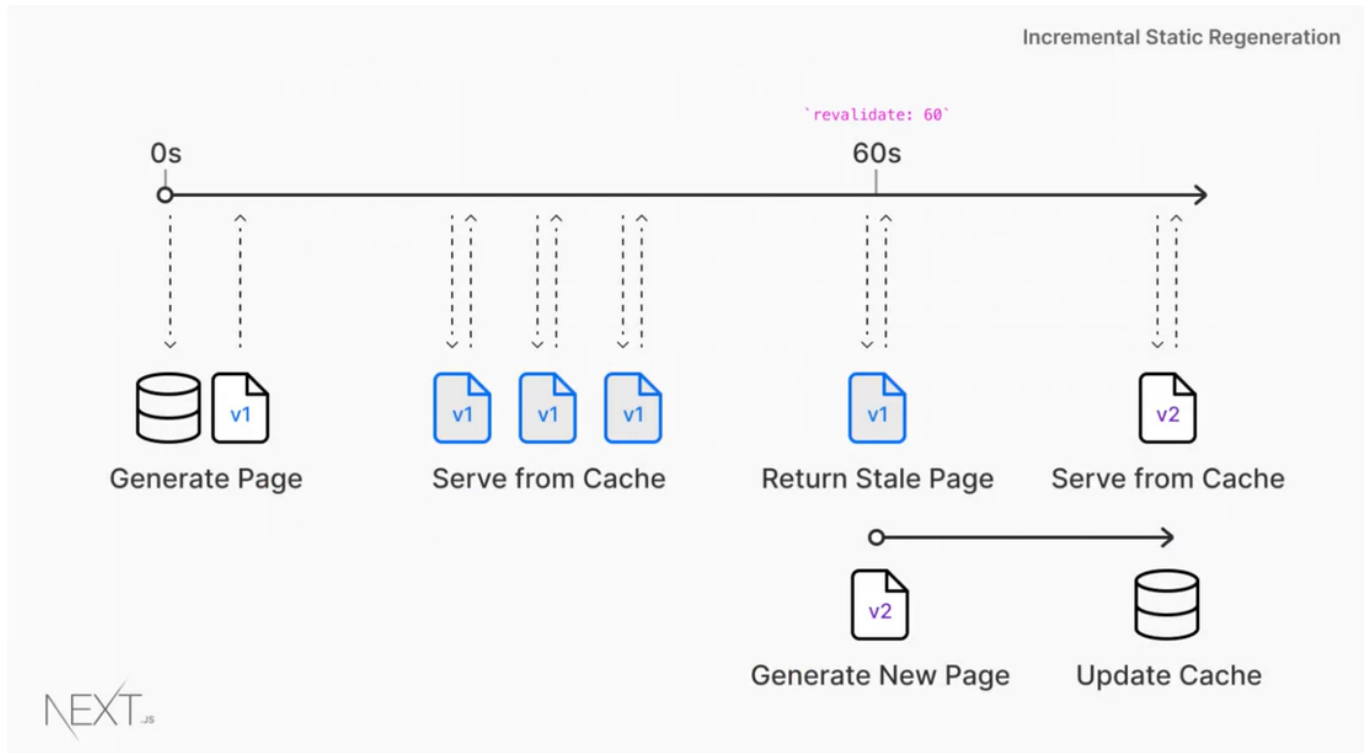
Например header или footer страницы отрендерить, там где не нужны данные отрендерить заранее.

App Shell — это минимальный набор HTML, CSS и JavaScript, необходимый для отрисовки UI-оболочки приложения (например, шапки, навигации, футера), **без данных**. Сейчас это очень распространённая концепция.

Можно просто выкинуть часть верстки за Реакт (в nextjs это не получится, но он умеет по компонентам делать сам). В Яндекс Поиске точно так делали.

ISR (Incremental Static Regeneration)

Это чисто Nextjs технология. Она довольно просто работает:



ISR это лучшее из двух миров: из SSR и SSG.

- Есть какие-то статические страницы (созданные также во время билда).
- Мы перерендериваем их по какому-то триггеру (по таймеру или другому процессу).
<https://nextjs.org/docs/app/guides/incremental-static-regeneration>

Incremental Static Regeneration позволяет:

- Генерировать страницы **на этапе билда**, как в SSG (Static Site Generation),
- Но при этом **обновлять их позже**, как в SSR (Server-Side Rendering), **без полного ребилда** всего сайта.

Это значит: ваши страницы могут быть статичными, но при этом «живыми» и обновляемыми.

SSI (Server Side Include)

SSI - это клиентский рендер, при котором часть логики выносится на сервер.

"то простая серверная технология для вставки содержимого одного файла в другой **на стороне сервера** перед тем, как HTML отдается клиенту.

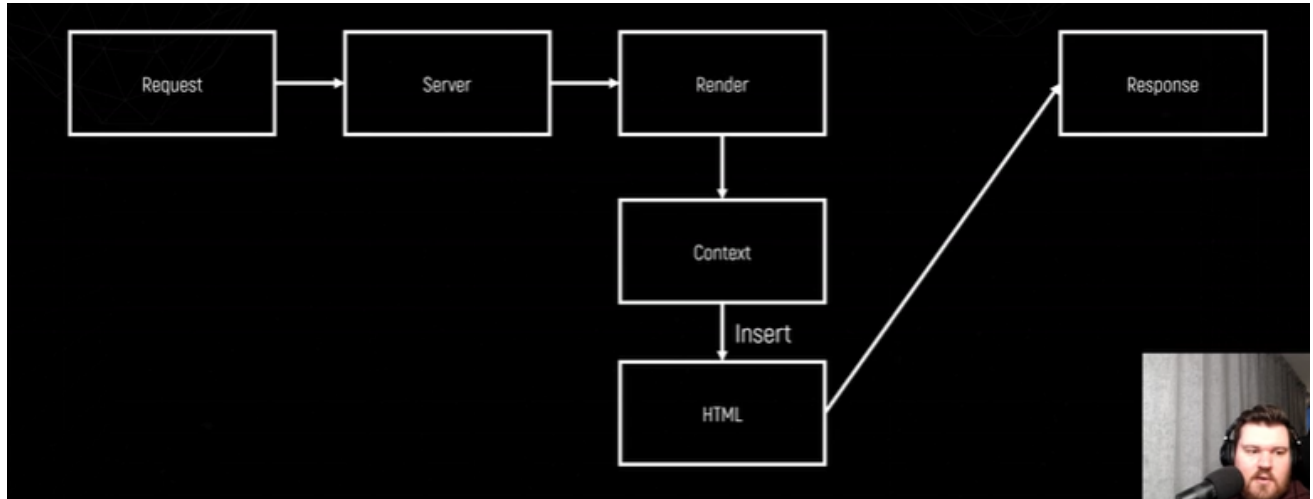
- Это технология только Nginx или Apache.
То есть Nginx позволял с помощью специальных выражений выполнять простую логику на стороне Nginx.

Пример:

```
<!--#include file="header.html" -->
```

Если эта строка находится в index.shtml или .shtml-файле, то сервер вставит содержимое header.html в этот момент.

- Совместимо только с HTML. Никакого JS или React.



Оптимизация процесса SSR

Вариантов нет, используем только SSR и оптимизируем его.

Из чего складывается время рендера страницы

- Загрузка HTML довольно история, она почти полностью зависит от рендера на сервере. Повлиять мы можем именно на серверный рендеринг, там нужны оптимизации другие (потом подробнее).
- Время парсинга HTML напрямую зависит от того, насколько много контента возвращается. Потенциально можно покапаться в стейте и часть стейта не отправлять, сократить количество тегов.
- Загрузка скриптов. Нужно стараться доставлять как можно меньше скриптов.
- Инициализация JS.
- Выполнение JS.
- Гидрация (hydration) - поле для экспериментов, можно делать очень много оптимизаций различных.

Кэширование

Что можно кэшировать:

1. Запросы.

2. Рендеры страниц (SSG).
3. Рендеры компонентов.

Кэширование запросов

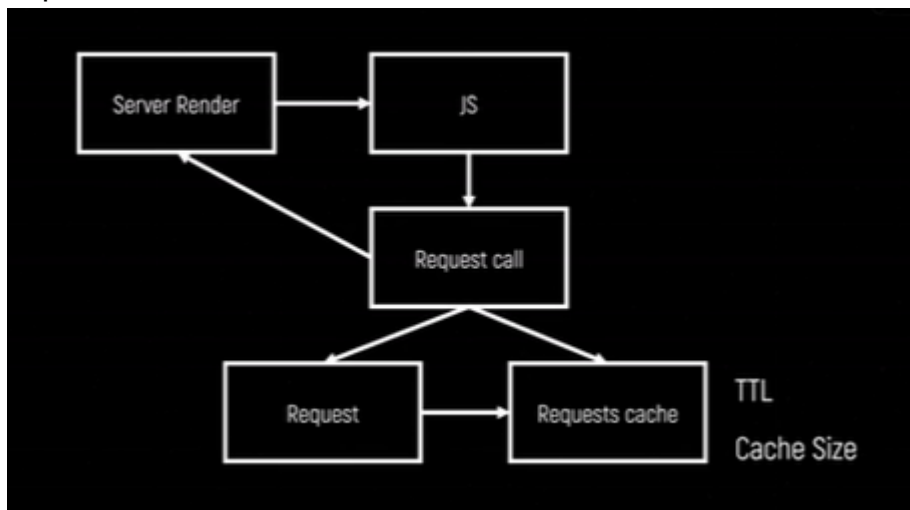
Понятно как кэшировать на клиенте, но что делать на сервере.

- Запускаем рендер, делаем JS
- Делаем какой-то запрос и есть развилка:

1. Честно сделать запрос.
2. Либо сделать запрос и положить в кэш.
3. Либо достать из кэшка.

Вот тут начинаются проблемы, во-первых некоторые запросы кэшировать нельзя, они очень персонализированы (например куки и авторизация). Кэшировать нужно осторожно.

Есть риск, кэширование запросов может потечь. Нужно думать как ограничить кэш и делать timeToLive (TTL) и размер кэша (size). Важно также учитывать персонализацию.



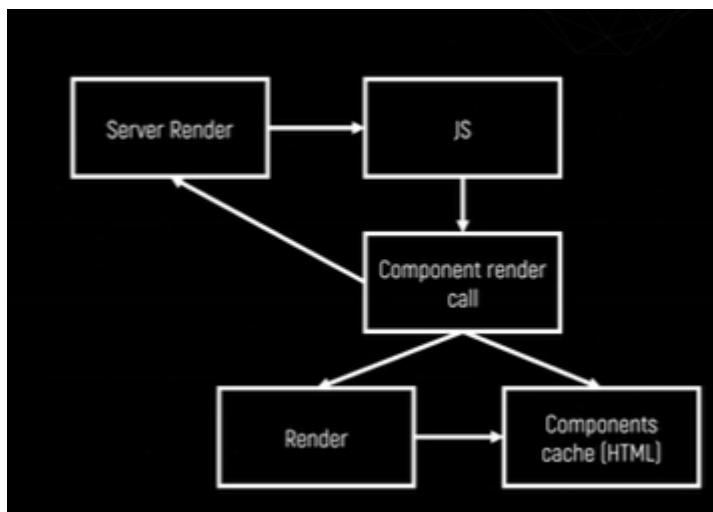
Кэширование компонентов

По сути серверные компоненты реакта (RSC) это все было создано чтобы кэшировать компоненты.

Вот как это сделано в tramvai

<https://github.com/tramvaijs/tramvai/blob/main/packages/libs/lazy-render/src/lazy-render.tsx>

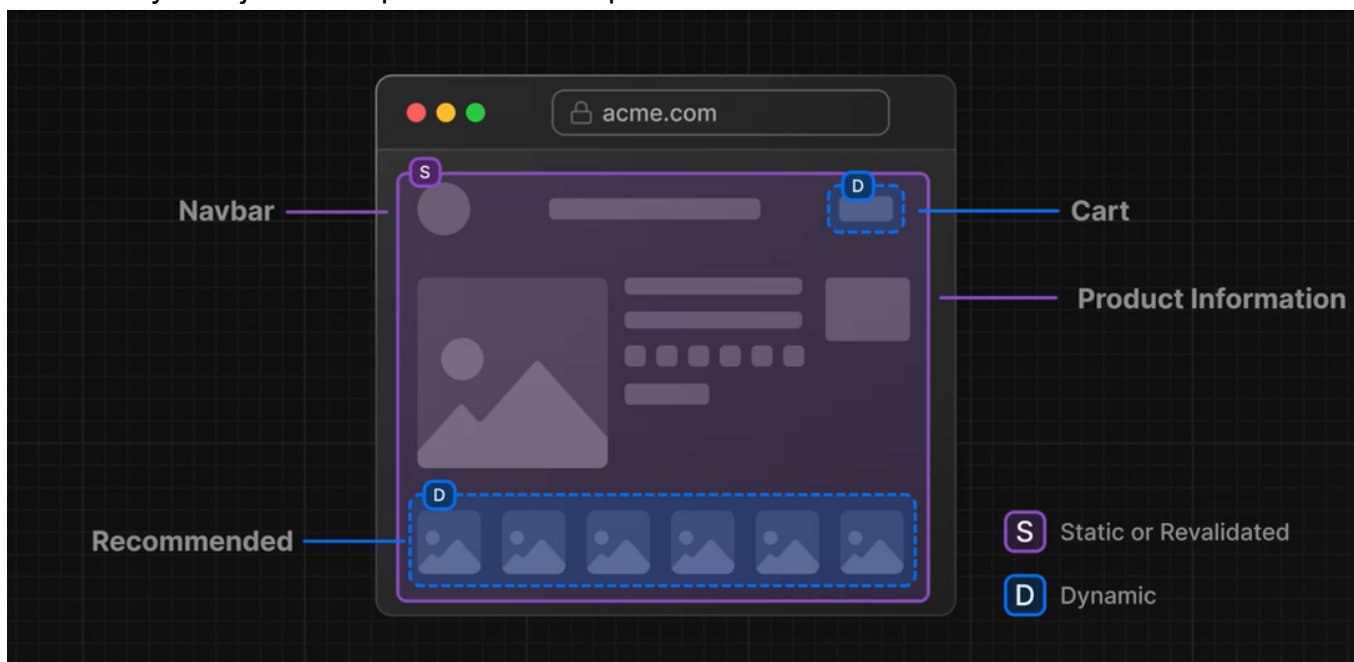
По сути мы либо честно рендерим компонент, либо рендерим и вставляем как статический HTML:



Но так не получится сделать если компонент зависит от пропсов или имеет стейт. Нужно тогда делать ключ кэша, чтобы он зависел от пропсов.

Partial prerendering

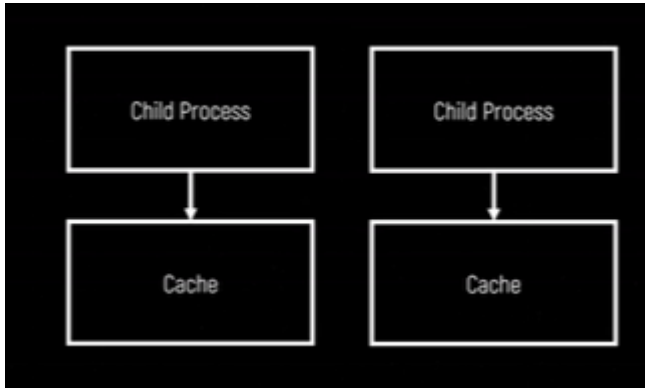
Она есть у Nextjs в экспериментальном режиме.



Работает так: делим страничку на части и какие-то рендерятся статически на этапе сборки и достаются из кэша, а какие-то в рантайме и мы все это вместе соединяем. Также там есть хитрая гидрация. То есть здесь соединяются все техники прям.

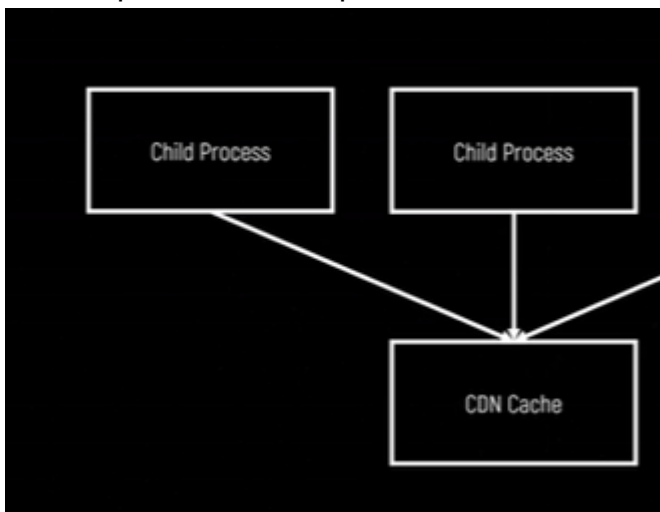
Разделяемый Кэш

Nodejs скейлится горизонтально. Но тут есть проблема: У каждого инстанса свой кэш:



Это справедливо как для кластер модуля ноды, так и для разных виртуалок. Чтобы шарить кэш между всем, нужно возиться с **Inter-process communication (IPC)**.

Есть вариант сделать разделяемый кэш в CDN:



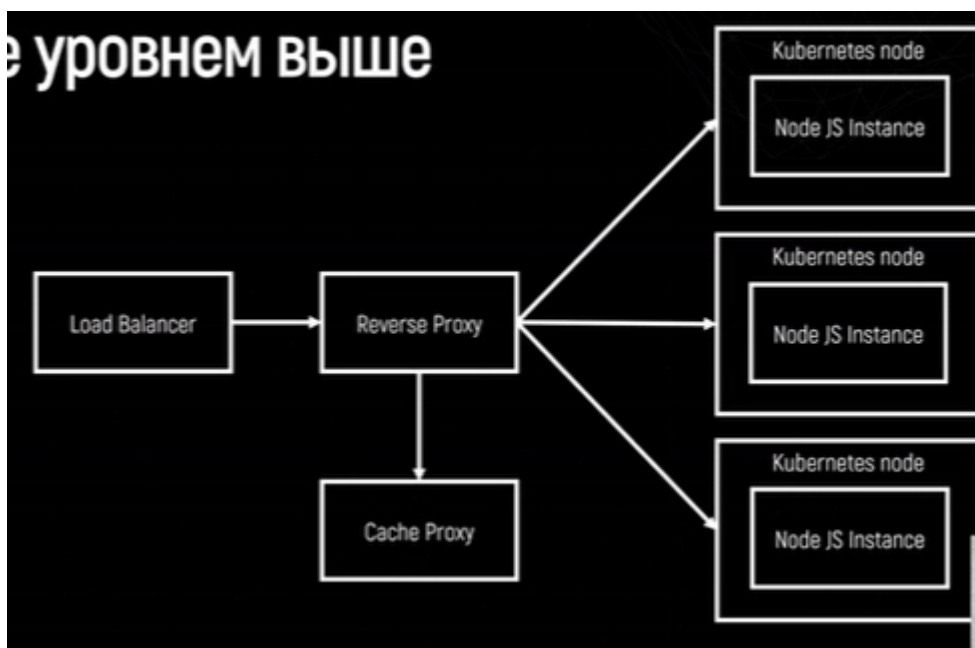
Кэширование уровнем выше

- <https://varnish-cache.org/>
- <https://docs.nginx.com/nginx/admin-guide/content-cache/content-caching/>
- <https://www.squid-cache.org/>

С точки зрения инфры выглядит так:



- Три инстанса ноды, реверс прокси, лoad балансер.
- Приходит запрос, перед тем как пойти за запросом в ноду, ставится еще один прокси с кэшем

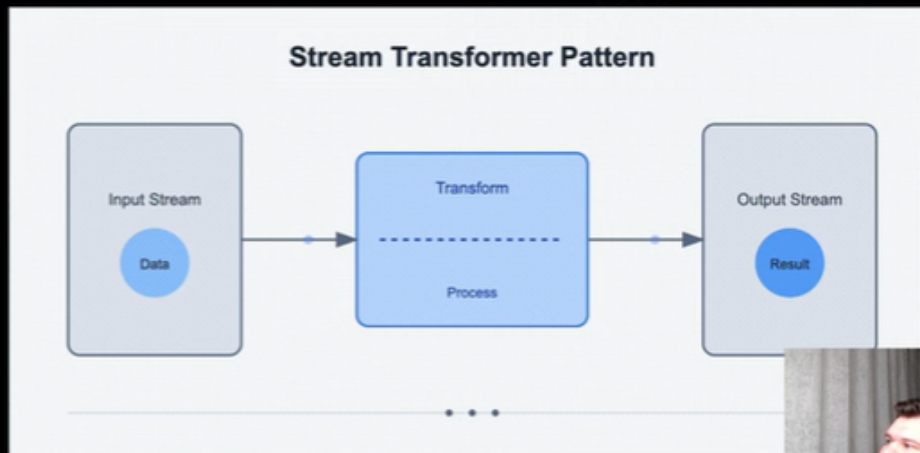


И он на некоторые запросы отдает уже закэшированные странички. Минусы в том, что такие кэши легко сломать: с помощью редиректов или из-за персональных данных.

Потоковый рендер

Потоковый рендер

NodeJS stream



<https://nodejs.org/api/stream.html>

Сейчас очень активно развивается эта концепция.

Потоки вообще не новая концепция, она идет еще с unix. Сейчас активно используется в серверном рендере.

Суть в том, что вместо всего HTML-документа с сервера отдается контент по кусочкам.

Пример папки `stream`: Есть огромный json-файл, на чтение которого нужно много памяти и ресурсов. Если говорим про передачу этот json по сети, то сначала его нужно полностью загрузить, а потом отправить.

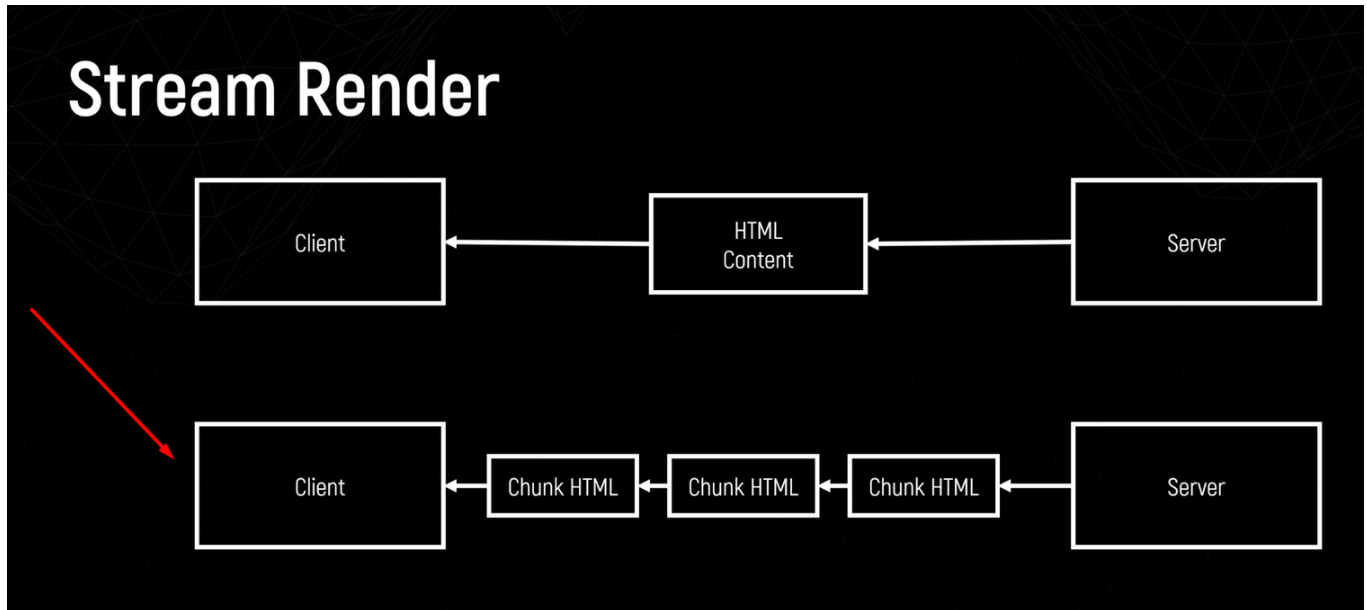
Решение - есть возможность сделать при чтении файла стрим. Если его запустить, то видно будет, что контент по честному читается маленькими чанками (16 байт). Сейчас это на запись в другой файл настроено, но можно сделать отправку по сети.

В случае стримов будет ли работать парсинг HTML? нужно ли его как-то по другому делать? - Проблем с этим вообще нет, браузер умеет работать с чанками, он может прилетевший кусочек сам зарендерить. На этом и базируются все оптимизации.

Хороший пример: ПОИСК в яндексе, там отправка чанками на уровне балансера. особенно заметно на медленном интернете.

Недостатки: метод `renderToPipeableStream` медленнее чем `renderToString`, сложная инфраструктура, иногда балансир или `nginx` не умеет этого. На маленьких проектах нет смысла.

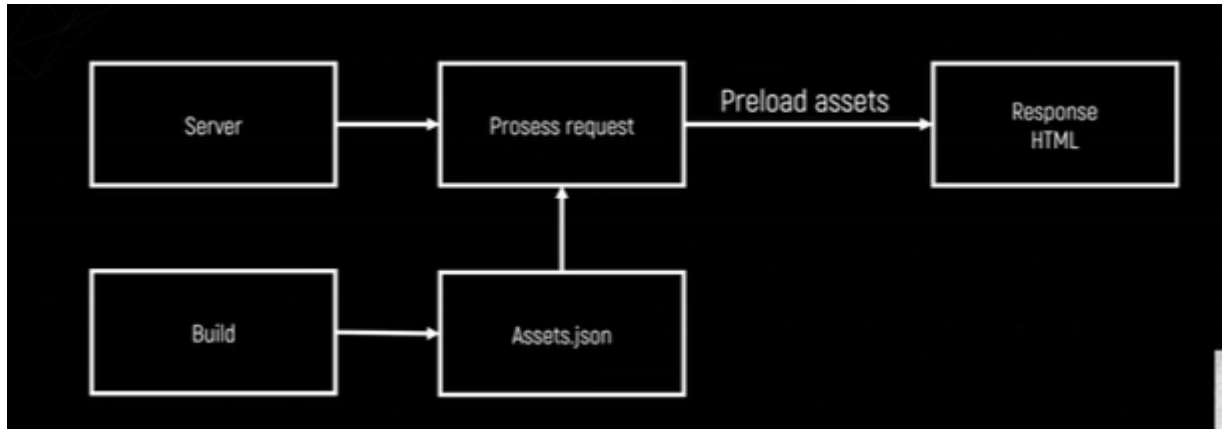
Пример схемы:



Важный нюанс: стримы не дружат с SEO (стрим HTML дружат), а вот именно гидрации. То есть если выключить JS на страничке и если страничка не рендрится, то поисковик ее не проиндексирует.

Предзагрузка/инлайн ассетов

Есть возможность при сборке генерировать `assets.json` - и сервер про этот файл знает:



Соответственно при обработке request достает из этого файла информацию о том какие ассеты на этой страничке нужны и может допустим их заpreloadить вставив специальные теги в html.

В VITE это `manifest.json`.

Например можно стили и js сразу инлайнить в html, потому что нет ничего быстрее чем инлайн стили и js. Хотя ударит по размеру html. Это самый быстрый способ выполнить этот код.

Оптимизация гидрации

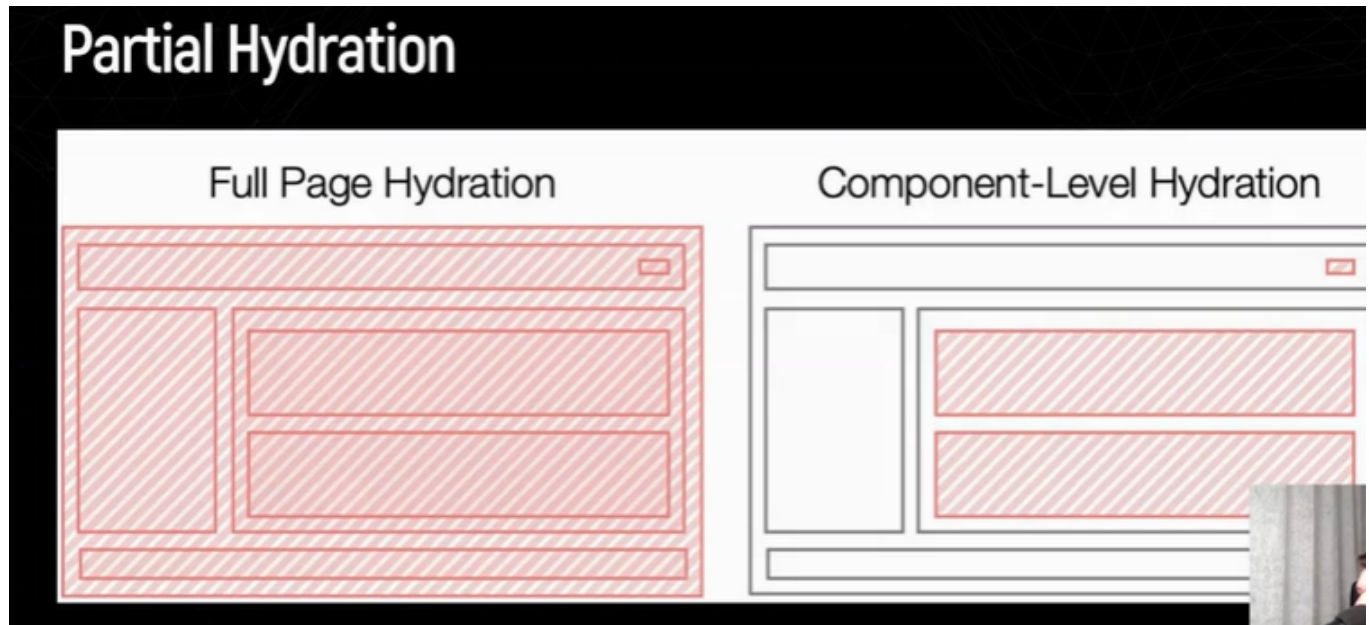
Это очень дорогой и сложный процесс, придумывают даже новые фреймворки, которые оптимизируют этот процесс.

Гидрация напрямую связана с количеством js.

Например на страничке много-много элемент и мы грузим для гидрации js-код, но на самом деле есть шапка и js который гидрирует тоже грузится, по сути js содержит шаблон этой шапки и вопрос, а надо ли вообще гидрировать? То есть зачем гидрировать шапку?

Partial Hydration

В результате появляется концепция частичной гидрации:



Тем самым много js не загружается, в эту сторону идет React с серверными компонентами.

Lazy Hydration

Популярно по Vue - ленивая гидрация:

The image illustrates the concept of lazy hydration in a web application. On the left, a screenshot of a mobile app interface for 'discount' shows an iPhone 13. On the right, a diagram compares the state of hydration 'before' and 'after' user interaction. The 'before' state shows the entire page body as interactive. The 'after' state shows only the top of the body as interactive, with a 'user scroll' timeline indicating that hydration occurs as the user scrolls. A small video inset of a person speaking is in the bottom right corner.

Зачем гидрировать все сразу? Например можно гидрировать кусочек страницы, чтобы работали клики табы и прочее только при скролле например. Например открыли страницу, гидрируется то что попало во вью-порт, а то что не попало гидрируется потом.

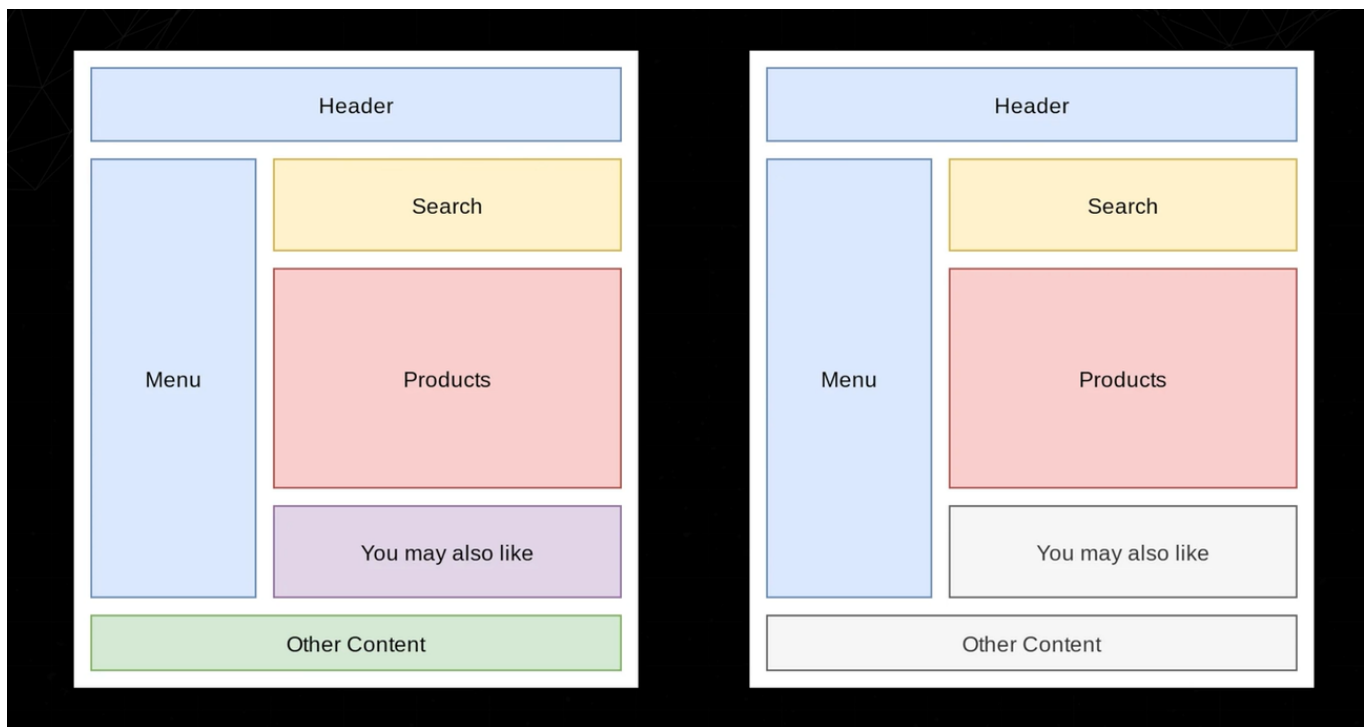
Она работает через Intersection Observer.

При нем отдается весь html, это чисто клиентская история, не мы не загружаем весь js сразу.

Progressive Hydration

Еще одна альтернатива lazy hydration - прогрессивная гидрация, она работает вместе с потоковым рендером, то есть отдается в потоке html и сразу рисуется контент, который пришел.

тут же идея что есть стриминг, отдаем шапку и тут же отдаем js для гидрации этой шапки :



И она сразу же гидрируется. Также она зависит еще от данных, поэтому какие-то элементы странички могут сгенерироваться гораздо позднее.

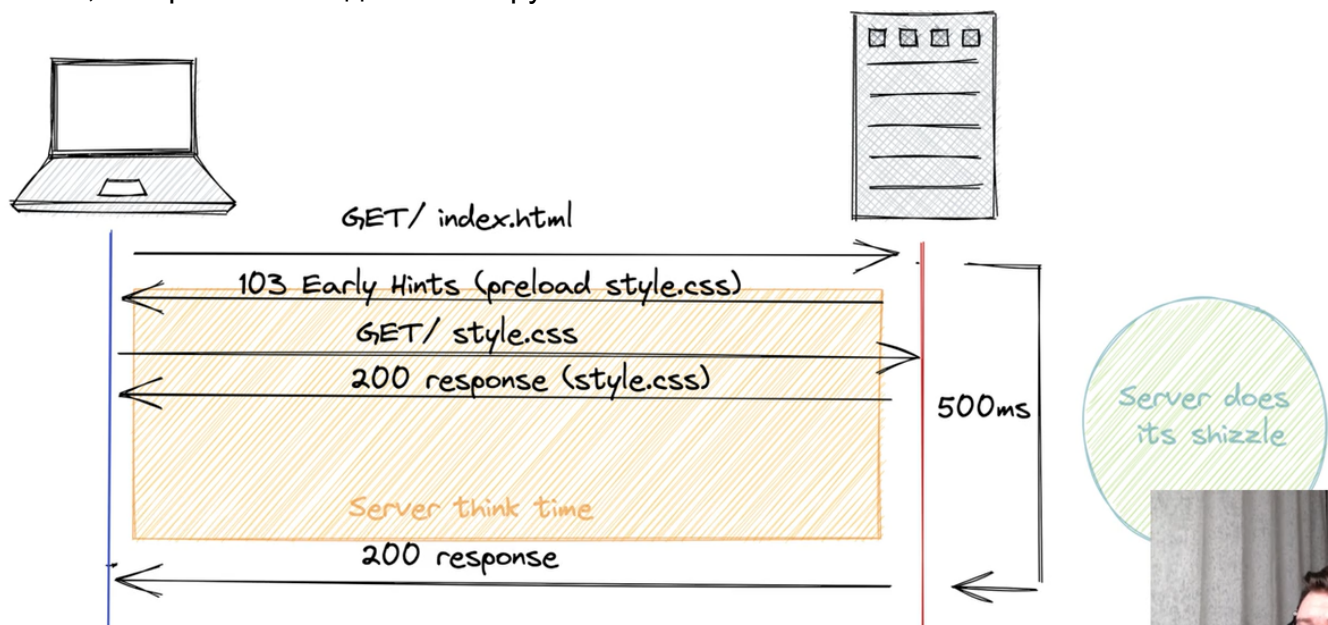
Это самая продвинутая технология.

Early Hints

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/103>

Это новый механизм, который еще мало где поддерживается.

Возможность на сервере отдать 103 статус ответа и можно в этом респонсе отдать список ассетов, которые клиент должен загрузить:



Например на сервере `renderToString` занимает 500мс, соответственно браузер 500мс просто ждет ответ и ничего не делает, НО можно вместо статуса ответа 200, вернуть 103 и

сказать "давай пока ждешь, сразу загрузишь style.css, основной код js". Это очень сильно экономит время.

Нюанс в том, что возвращается 103 респонс и браузер не разрывает соединение, а дальше ждет ответа 200 либо другого статуса.

Прям хорошая технология.

Tramvai точно поддерживает эту технологию.

Альтернативные подходы SSR

HTMX

Это такой подход, где вообще не пишется никакой js. Весь js пишется в html тегах на сервере.

<https://htmx.org/>

<https://htmx.org/examples/infinite-scroll/>

Это история про честный тонкий клиент.

HotWite

<https://hotwired.io/>

Такой же подход

- гибче чем htmx
- Продолжение ruby on rails.

Qwik

Astro

Marko